

Shogird

Платформа стажировок для студентов и IT-компаний Узбекистана

Проект по курсу «Big Data»: база данных и сайт (PostgreSQL + MongoDB)

Авторы	[Имя Фамилия, Имя Фамилия]
Группа, курс	[группа, курс]
Дата	3 ноября 2026
Публичный сайт	https://shogird.onrender.com
Код (GitHub)	https://github.com/atmtab/shogird

Первый запрос к сайту после паузы занимает до минуты: бесплатный тариф Render «засыпает» без посетителей.

1. Бизнес-идея

Проблема в Узбекистане

- **Студенты** не знают, где искать стажировку и каких навыков им не хватает. Отклики в Telegram-каналах и на общих job-сайтах остаются без ответа и без статуса: студент шлёт десятки писем и не понимает, на каком этапе он.
- **Компании** тратят время на неподходящих кандидатов, а время собеседований согласуют вручную в переписке.
- **Вузы** не видят, каких навыков рынок ждёт от выпускников.

Решение: Shogird

1. Каталог стажировок с поиском и фильтрами (направление, город, формат, оплачиваемость, навык, «резидент IT Park»).
2. Профиль навыков студента и **процент совпадения** с каждой вакансией.
3. Прозрачная воронка заявки: *подана → в шорт-листе → собеседование → оффер → принят*.
4. Запись на собеседование **по слотам**: компания публикует время, студент выбирает, двойная бронь невозможна.
5. Аналитика **«разрыв навыков»**: что ищут студенты, что требуют вакансии и что реально умеют студенты.

Целевые пользователи

Кто	Что получает
Студент вуза	Единое место поиска, понятный статус заявки, подсказка, какой навык подтянуть
IT-компания (в том числе резидент IT Park)	Кандидаты, отсортированные по совпадению навыков, запись на собеседование без переписки
Вуз (следующий этап)	Отчёт о востребованности навыков для обновления программ

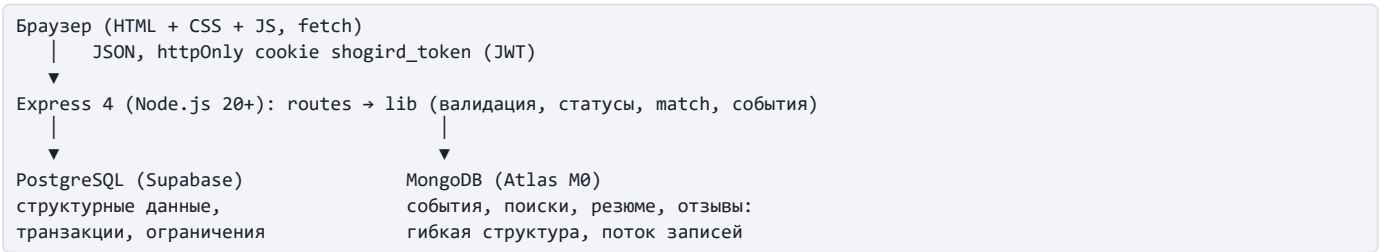
Как продукт зарабатывает

- Платное продвижение вакансий (вакансия выше в каталоге).
- Подписка для компаний: больше активных вакансий и расширенная аналитика по кандидатам.
- Платные отчёты для вузов о востребованности навыков (данные «разрыва навыков»).

Конкуренты

Конкурент	Чем отличается Shogird
hh.uz	Общий рынок труда, не заточен под стажёров и студентов: нет процента совпадения с профилем студента, нет записи по слотам и аналитики разрыва навыков.
Telegram-каналы с вакансиями	Нет статусов заявки, фильтров, истории и аналитики; отклик уходит «в пустоту».

2. Архитектура



Хостинг: Render (приложение), Supabase (PostgreSQL через Session pooler), MongoDB Atlas M0. Фронтенд отдаётся тем же Express-сервером как статические файлы: без сборки и без фреймворков.

Почему две базы

PostgreSQL: структурные данные	MongoDB: события и документы
users, universities, students, companies, skills, student_skills, vacancies, vacancy_skills, applications, interview_slots, interviews. Нужны связи, уникальность, внешние ключи, **транзакции** (бронь слота, последнее место) и ограничения целостности.	events, search_logs, resumes, internship_reviews. Нужны быстрые записи потока событий, гибкая структура (резюме у каждого студента разное, фильтры поиска разные) и агрегации (`$unwind`, `$group`).

Числовые ID в Mongo (studentId, vacancyId, companyId) совпадают с ID в PostgreSQL, поэтому данные двух баз соединяются на уровне приложения. Пример: «разрыв навыков» берёт спрос на навык из Mongo (поисковые логи), а число вакансий и студентов из PostgreSQL.

Принципы реализации

- SQL только параметризованный (`$1, $2…`), защита от SQL-инъекций. Деньги в `NUMERIC(12,2)`.
- Проверка и изменение одним запросом (`UPDATE … WHERE seats_left > 0`), без гонок между `SELECT` и `UPDATE`.
- События аналитики пишутся в Mongo **после** `COMMIT` и без `await`: откат транзакции не оставляет событий, а сбой логирования не ломает ответ.
- Пароли хранятся как bcrypt-хэш (cost 10). Токен в httpOnly cookie, поэтому недоступен скриптам страницы.
- Интерфейс на трёх языках (RU, UZ, EN) без перезагрузки страницы.

3. Проектирование базы данных

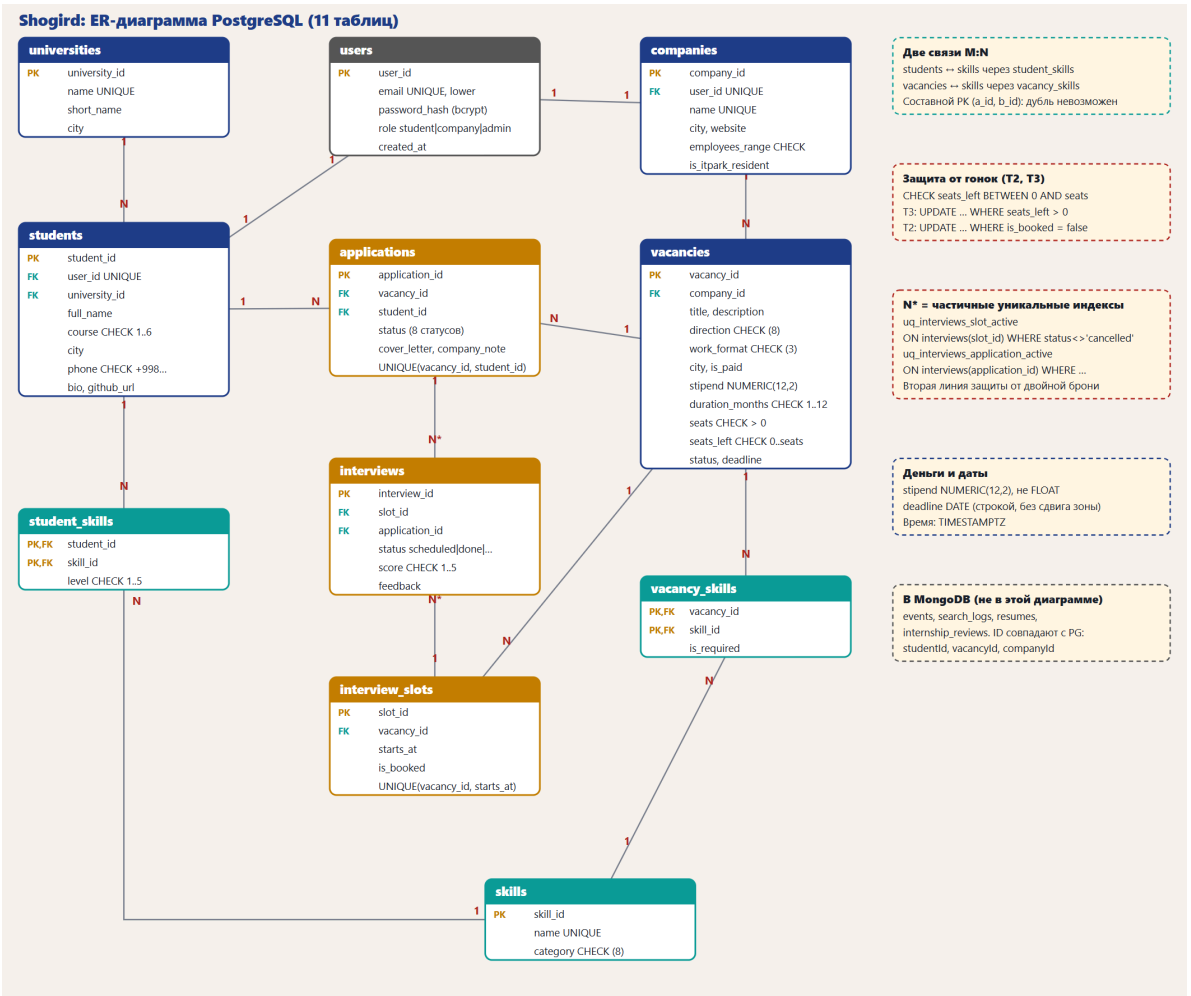


Рисунок 1. ER-диаграмма PostgreSQL (11 таблиц, две связи M:N)

Таблицы

Таблица	Назначение
users	Учётные записи: email (уникальный, в нижнем регистре), bcrypt-хэш, роль student/company/admin
universities, students, companies	Справочник вузов; профиль студента (вуз, курс, город, телефон); профиль компании (резидент IT Park, размер)
skills	Справочник из 42 навыков по 8 категориям
student_skills (M:N)	Навыки студента с уровнем 1–5, составной PK (student_id, skill_id)
vacancies	Стажировка: направление, формат, стипендия, места (seats, seats_left), статус, дедлайн
vacancy_skills (M:N)	Требуемые навыки вакансии; флаг is_required: обязательный или «будет плюсом»
applications	Заявка студента на вакансию, 8 статусов
interview_slots, interviews	Слоты времени от компании; бронь слота заявкой, оценка и отзыв о собеседовании

Ограничения целостности

- `CHECK (seats_left BETWEEN 0 AND seats)`: мест не может быть меньше нуля или больше, чем всего. Последняя линия защиты для сценария «последнее место».
- `CHECK (is_paid OR stipend IS NULL)` и `stipend >= 0`: стипендия только у оплачиваемых стажировок.
- `UNIQUE (vacancy_id, student_id)`: одна заявка студента на вакансию. `UNIQUE (vacancy_id, starts_at)`: один слот на время.
- **Частичные уникальные индексы** `uq_interviews_slot_active` и `uq_interviews_application_active` (`WHERE status <> 'cancelled'`): слот и заявка не могут иметь двух активных собеседований, а отменённое освобождает их. Это вторая линия защиты от двойной брони после `UPDATE … WHERE is_booked = false`.
- `CHECK` на роли, статусы, направления, формат, курс 1–6, оценку 1–5 и формат телефона `+998XXXXXXXXX`.

Нормализация

- Связи «многие ко многим» вынесены в таблицы-связки, а не в массивы: студент ↔ навык и вакансия ↔ навык (3НФ, навык хранится один раз).
- Справочники (universities, skills) отдельно, поэтому название вуза и навыка не дублируется.
- `seats_left` хранится в вакансии как счётчик, а не считается через `COUNT` по заявкам: это сознательная денормализация ради атомарного `UPDATE` и быстрого чтения каталога. Целостность обеспечивает `CHECK`.
- Индексы на внешних ключах и частых фильтрах: `(status, direction)`, `city`, `(vacancy_id, status)` у заявок и другие.

4. Запросы и аналитика

Ниже 5 из 20 SQL-запросов (`sql/03_queries.sql`) и 3 из 12 запросов MongoDB (`mongo/02_queries.mongodb.js`). Результаты получены на сид-данных: 26 открытых вакансий, 40 студентов, 12 компаний, 60 заявок.

PostgreSQL

Q1 (Q11 в файле). Какие компании получили больше 5 заявок?

> Бизнес-вопрос: кто самые востребованные работодатели? Им можно предложить подписку. Приём: JOIN трёх таблиц, GROUP BY, HAVING.

```
SELECT c.name, count(a.application_id)::int AS applications
FROM companies c
JOIN vacancies v     ON v.company_id = c.company_id
JOIN applications a  ON a.vacancy_id = v.vacancy_id
GROUP BY c.company_id, c.name
HAVING count(a.application_id) > 5
ORDER BY applications DESC
```

name	applications
Registan Labs	14
Chorsu Digital	8
Silk Code	7
Navoiy Soft	6
Bulut Tech	6

Q2 (Q15). Самые популярные вакансии внутри каждой компании

> Бизнес-вопрос: какие стажировки привлекают больше всего заявок, а какие стоит продвигать? Приём: оконная функция RANK() OVER (PARTITION BY ...).

```
SELECT c.name AS company, v.title, count(a.application_id)::int AS applications,
       RANK() OVER (PARTITION BY c.company_id ORDER BY count(a.application_id) DESC) AS rank_in_company
FROM vacancies v
JOIN companies c ON c.company_id = v.company_id
LEFT JOIN applications a ON a.vacancy_id = v.vacancy_id
GROUP BY c.company_id, c.name, v.vacancy_id, v.title
ORDER BY c.name, rank_in_company, v.title
```

company	title	applications	rank_in_company
Amudaryo Data	Machine Learning Intern	4	1
Amudaryo Data	Power BI Analyst Intern	0	2
Andijan Systems	PHP Backend Intern	1	1
Andijan Systems	QA Automation Intern	1	1
Bukhara Byte	Project Manager Intern	2	1
Bukhara Byte	Full-Stack Intern	0	2

Показаны первые 6 строк из 30.

Q3 (Q12). Сколько в среднем платят по городам и направлениям?

> Бизнес-вопрос: какая вилка стипендий на рынке? Студент видит ориентир, а компания сравнивает своё предложение. Приём: GROUP BY по двум полям, агрегат avg.

```
SELECT city, direction, count(*)::int AS vacancies, round(avg(stipend))::float AS avg_stipend_uzs
FROM vacancies
WHERE status = 'open' AND is_paid
GROUP BY city, direction
ORDER BY avg_stipend_uzs DESC
LIMIT 10
```

city	direction	vacancies	avg_stipend_uzs
Tashkent	devops	1	4500000
Tashkent	data	3	4166667
Urgench	data	2	3700000
Tashkent	backend	3	3600000
Tashkent	mobile	2	3300000

Q4 (Q19). Процент совпадения вакансий со студентом (match score)

> Бизнес-вопрос: какие стажировки лучше всего подходят Азизе? Обязательный навык весит 1, «будет плюсом» 0,5. Этот запрос лежит в основе рекомендаций и сортировки кандидатов.

```
SELECT v.vacancy_id, v.title,
       ROUND(100.0 * SUM(CASE WHEN ss.skill_id IS NOT NULL
                              THEN CASE WHEN vs.is_required THEN 1 ELSE 0.5 END ELSE 0 END)
                   / SUM(CASE WHEN vs.is_required THEN 1 ELSE 0.5 END))::int AS match_pct
FROM vacancies v
JOIN vacancy_skills vs ON vs.vacancy_id = v.vacancy_id
LEFT JOIN student_skills ss ON ss.skill_id = vs.skill_id AND ss.student_id = 1
WHERE v.status = 'open'
GROUP BY v.vacancy_id, v.title
ORDER BY match_pct DESC, v.vacancy_id
LIMIT 10
```

vacancy_id	title	match_pct
2	Frontend Intern (React)	100
5	React Developer Intern	100
23	Frontend Vue Intern	67
27	iOS Swift Intern	50
29	Full-Stack Intern	50

Показаны первые 5 строк из 10.

Q5 (Q20). Спрос вакансий и предложение студентов по навыкам

> Бизнес-вопрос: каких навыков не хватает на рынке? Это основа дашборда «разрыв навыков»: Git требуют 15 вакансий, а владеют им лишь 6 студентов. Приём: JOIN, коррелированный подзапрос, DISTINCT.

```
SELECT s.name AS skill,
       count(DISTINCT vs.vacancy_id)::int AS open_vacancies,
       (SELECT count(*) FROM student_skills ss WHERE ss.skill_id = s.skill_id)::int AS students_with_skill
FROM skills s
JOIN vacancy_skills vs ON vs.skill_id = s.skill_id
JOIN vacancies v       ON v.vacancy_id = vs.vacancy_id AND v.status = 'open'
GROUP BY s.skill_id, s.name
ORDER BY open_vacancies DESC, s.name
LIMIT 10
```

skill	open_vacancies	students_with_skill
Git	15	6
SQL	9	6
Docker	6	5
Python	5	10
Excel	3	0

MongoDB

M1 (Q6). Воронка: сколько событий каждого типа?

> Бизнес-вопрос: сколько просмотров превращается в отклики, брони и принятые офферы? Агрегация $group + $sort.

```
db.events.aggregate([
  { $group: { _id: '$type', events: { $sum: 1 } } },
  { $sort: { events: -1 } }
])
```

type	events
vacancy_view	42 024
apply	8 927
slot_booked	4 155
company_view	2 969
offer_accepted	1 925

M2 (Q8). Какие навыки студенты ищут чаще всего?

> Бизнес-вопрос: на какие навыки есть спрос у студентов и находят ли они то, что ищут? Поле avgResults показывает среднее число найденных вакансий: у UI/UX всего 0,3, то есть студенты ищут, а предложения мало. Приём: $unwind по массиву filters.skills.

```
db.search_logs.aggregate([
  { $unwind: '$filters.skills' },
  { $group: { _id: '$filters.skills', searches: { $sum: 1 }, avgResults: { $avg: '$results' } } },
  { $sort: { searches: -1 } }, { $limit: 10 }
])
```

skill	searches	avgResults
Python	2 977	1.3
React	2 542	0.8
UI/UX	2 180	0.3
JavaScript	1 484	0.9
Figma	1 254	0.4

М3 (Q10). У каких компаний лучший средний рейтинг?

> Бизнес-вопрос: каким работодателям доверяют стажёры? Данные идут в блок «Топ компаний» и в профиль компании. Агрегация $group по companyId (ID совпадает с PostgreSQL).

```
db.internship_reviews.aggregate([
  { $group: { _id: '$companyId', avgRating: { $avg: '$rating' }, reviews: { $sum: 1 } } },
  { $sort: { avgRating: -1, reviews: -1 } }, { $limit: 5 }
])
```

companyId	avgRating	reviews
11	4.67	3
6	4.5	2
9	4	5
5	4	2
2	4	1

Объединение двух баз: «разрыв навыков»

Эндпоинт `GET /api/stats/skills-gap` считает по каждому навыку три числа: сколько раз его искали (Mongo), сколько открытых вакансий его требуют и сколько студентов им владеют (PostgreSQL). «Дефицит» равен спросу вакансий минус предложению студентов. На дашборде это три полосы на навык; именно эта связка двух баз показывает смысл их совместного использования.

5. Эксперимент с большими данными

Скрипт `sql/04_big_data.sql` создаёт архивную таблицу **applications_archive на 500 000 строк** детерминированной генерацией (`generate_series` и хэши, поэтому результат воспроизводим) с сезонностью: май–июль по 45–47 тысяч заявок в месяц, остальные месяцы около 12 тысяч.

PostgreSQL: «backend-заявки в Ташкенте за 30 дней»

```
EXPLAIN ANALYZE
SELECT count(*) FROM applications_archive
WHERE direction = 'backend' AND city = 'Tashkent'
  AND applied_at >= now() - interval '30 days';

CREATE INDEX idx_archive_dir_city_date
  ON applications_archive (direction, city, applied_at);
```

	План выполнения	Время выполнения
До индекса	Parallel Seq Scan: читается вся таблица (500 000 строк)	**68,4 мс**
После индекса (direction, city, applied_at)	Bitmap Index Scan: читаются только подходящие строки	**1,0 мс**

Пояснение. Без индекса планировщик вынужден просмотреть все строки. Составной индекс начинается с двух равенств (`direction`, `city`), а затем диапазон по дате, поэтому по нему можно сразу перейти к нужному участку. Запрос выбирает малую часть таблицы (одна комбинация направления и города за 30 дней), поэтому индекс выгоден: если бы запрос возвращал большую долю таблицы, планировщик всё равно выбрал бы Seq Scan. Время зависит от железа и кэша (при трёх прогонах до индекса 65–150 мс, после 1–3 мс), но порядок ускорения повторяется.

Цена индекса. Индекс ускоряет чтение, но замедляет вставки и обновления (индекс надо поддерживать) и занимает место на диске. Поэтому индексы стоят на частых фильтрах и внешних ключах, а не на всех колонках.

MongoDB: «отклики (apply) по направлению backend»

```
db.events.find({ direction: 'backend', type: 'apply' }).explain('executionStats')
db.events.createIndex({ direction: 1, type: 1 })
```

	План	Просмотрено документов	Время
До индекса	IXSCAN по {type, ts}, затем фильтр по direction	8 896	**17 мс**
После индекса {direction, type}	IXSCAN по составному ключу	1 145	**4 мс**

Составной индекс по обоим полям запроса сократил число просмотренных документов в 7,8 раза (с 8 896 до 1 145), потому что отбор целиком выполняется внутри индекса. В коллекции 60 000 событий; на таком объёме разница в миллисекундах невелика, но растёт вместе с коллекцией.

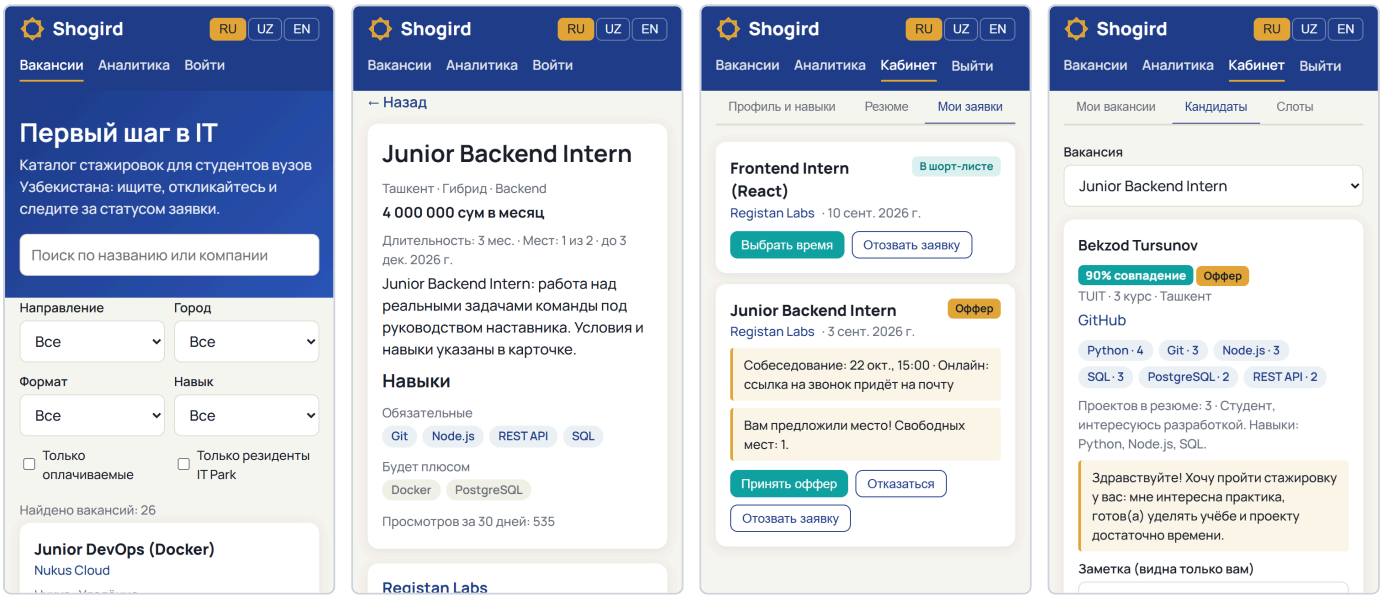
Состояние данных

База	Объём
PostgreSQL (сид)	53 пользователя, 30 вакансий (26 открытых), 60 заявок, 47 слотов, 42 навыка
PostgreSQL (эксперимент)	applications_archive: 500 000 строк
MongoDB	events 60 000, search_logs 30 000, resumes 40, internship_reviews 40 (около 18 МБ с индексами)

6. Сайт и API

Страницы

Страница	Кто	Содержимое
index.html	все	Каталог, поиск, фильтры, карточки с процентом совпадения, пагинация
vacancy.html, company.html	все	Описание, навыки (свои подсвечены), места, форма отклика; профиль компании и отзывы
dashboard.html	все	KPI, разрыв навыков, воронка, города, активность, топ компаний
auth.html	гость	Вход и регистрация (студент или компания)
student.html	студент	Профиль и навыки, резюме, заявки (бронь слота, оффер, отзыв), рекомендации
company-panel.html	компания	Вакансии, кандидаты по воронке, слоты собеседований



Каталог (RU) Карточка вакансии Заявки студента Кандидаты компании

Рисунок 2. Сайт на ширине 360 px (снимки публичного сайта)

API (34 маршрута, базовый путь /api)

Метод и URL	Назначение
GET /health, /meta	Состояние обеих баз; справочники для форм
GET /vacancies, /vacancies/:id, /companies/:id	Каталог с фильтрами (12 на страницу, поиск пишется в Mongo), карточки
POST /auth/register, /auth/login, /auth/logout; GET /auth/me	Регистрация (T1), вход (cookie), выход, текущий пользователь
GET, PUT /me/profile; PUT /me/skills; GET, PUT /me/resume	Профиль, навыки (T5), резюме в Mongo
GET /me/recommendations; POST /vacancies/:id/apply; GET /me/applications	Топ-10 по совпадению, отклик, мои заявки
GET /applications/:id/slots; POST …/book, …/accept, …/decline, …/withdraw	Свободные слоты, бронь (T2), принять оффер (T3), отказ, отзыв заявки
POST /companies/:id/reviews	Отзыв о стажировке (только после принятого оффера)
GET, POST /company/vacancies; PUT, DELETE /company/vacancies/:id	Вакансии компании (T4); DELETE закрывает, а не удаляет
GET /company/vacancies/:id/applications; PATCH /company/applications/:id	Кандидаты с процентом совпадения; смена статуса, заметка, оценка
GET, POST /company/vacancies/:id/slots; DELETE /company/slots/:id	Слоты собеседований
GET /stats/overview, skills-gap, funnel, cities, activity, top-companies	Аналитика из PostgreSQL и MongoDB

Ошибки возвращаются единым форматом `{ "error", "code" }`; код переводится на язык пользователя. Компания видит только свои сущности: на чужой ID отвечает 404.

Транзакции

Каждая транзакция следует шаблону: `BEGIN` → шаги → `COMMIT`; при ошибке `ROLLBACK` и повторный `throw`; в `finally` соединение возвращается в пул.

№	Операция	Что делает
T1	Регистрация	INSERT users + INSERT students/companies. Дубликат email: откат, ответ 409 EMAIL_TAKEN
T2	**Бронь слота**	SELECT заявки FOR UPDATE → атомарный UPDATE слота (is_booked = false) → INSERT interviews → статус «собеседование». Слот занят: откат, 409 SLOT_TAKEN
T3	**Принять оффер**	Заявка FOR UPDATE → UPDATE мест (seats_left > 0) → «принята» → остальные активные заявки студента «отозваны», будущие собеседования отменены, слоты освобождены. Мест нет: откат, 409 NO_SEATS
T4	Вакансия	INSERT/UPDATE vacancies + замена vacancy_skills одним блоком
T5	Навыки студента	DELETE + INSERT student_skills одним блоком

Код, которым гордимся: ключевой шаг T3

```
UPDATE vacancies
SET seats_left = seats_left - 1,
    status = CASE WHEN seats_left = 1 THEN 'closed' ELSE status END
WHERE vacancy_id = $1 AND seats_left > 0
RETURNING seats_left;
-- 0 затронутых строк => ROLLBACK и ответ 409 NO_SEATS
```

Почему проверка внутри UPDATE, а не отдельным SELECT. Между `SELECT` и `UPDATE` другой запрос может занять последнее место. Условие `seats_left > 0` внутри `UPDATE` проверяется и применяется атомарно; `FOR UPDATE` сериализует параллельные действия над одной заявкой. Если студент принимает последнее место одновременно с другим, один получает 200, второй 409 NO_SEATS, а данные не портятся. Проверено автоматическим тестом гонки: два параллельных запроса дают `[200, 409 NO_SEATS]`. Дополнительно `CHECK (seats_left BETWEEN 0 AND seats)` не даст базе принять некорректное значение, даже если ошибиться в коде.

Демо-сценарии

- **D1.** Азиза выбирает слот собеседования: статус «собеседование» (T2).
- **D2.** На Junior Backend Intern осталось 1 место, у Азизы и Бекзода статус «оффер». Бекзод принимает, Азиза получает «Мест больше нет», данные не меняются.
- **D3.** Попытка забронировать занятый слот: 409 SLOT_TAKEN.
- **D4.** Компания двигает кандидатов по воронке: шорт-лист, оффер, отказ.

7. Деплой и тестирование

Хостинг

Компонент	Сервис	Настройка
Приложение	Render (Web Service, Free)	Root Directory `server`, Build `npm install`, Start `npm start`; секреты в переменных окружения
PostgreSQL	Supabase (Free)	Session pooler (IPv4, порт 5432), `PGSSL=true`
MongoDB	Atlas M0 (Free)	Пользователь с паролем из букв и цифр, Network Access `0.0.0.0/0`

Секреты (пароли баз, JWT_SECRET) хранятся только в переменных окружения и в локальном `.env`, которого нет в git. Поиск по всей истории репозитория не нашёл облачных паролей. Скрипты `db:reset` и `seed:mongo` отказываются работать с не-localhost без явного флага `--cloud`, чтобы нельзя было случайно стереть облачную базу.

Проблемы, которые мы решили

Проблема	Причина	Решение
Хост Supabase не находится (ENOTFOUND)	Direct connection доступен только по IPv6	Session pooler: host … pooler.supabase.com, логин postgres.<project-ref>
Сайт отвечает, но все запросы к БД дают 500	Опечатка в `DATABASE_URL` в Render (не тот хост и имя базы)	Исправлена переменная и сервис перезапущен
PostgreSQL работает, а в Mongo «пусто» (events7d = null)	В Atlas не было записи Network Access 0.0.0.0/0; ошибки Mongo в приложении намеренно глушатся	Добавлен 0.0.0.0/0; health проверяет обе базы
Дедлайн вакансии сдвигался на день	Драйвер pg превращал DATE в JS-дату в часовом поясе UTC+5	Парсер типа DATE возвращает строку «ГГГГ-ММ-ДД»
Красная ошибка 401 в консоли у гостя	Страница спрашивает «кто я» и получает 401	`/auth/me?optional=true` отвечает 200 null для гостя

Чек-лист тестирования

Пункт	Результат
Поиск и фильтры возвращают верные результаты	Пройдено (браузер, три языка)
Карточка показывает данные из обеих баз	Пройдено: места и условия из PostgreSQL, просмотры и рейтинг из Mongo
Бронирование работает	Пройдено (сценарий D1 в браузере, тест API)
Дружелюбные ошибки (нет мест, неверный пароль, пустая форма, пустая выдача)	Пройдено
Дашборд показывает числа из агрегаций	Пройдено
В консоли браузера нет красных ошибок	Пройдено на всех страницах и языках
Нет паролей в GitHub	Пройдено (поиск по файлам и всей истории)
README объясняет запуск	Пройдено
Ширина 360 px без горизонтального скролла	Пройдено: 36 сочетаний «страница × язык»
Автоматические проверки	smoke: 144 проверки API; i18n:check: 236 ключей × 3 языка

Главная страница на реальном телефоне через мобильный интернет: [отметить после проверки] .

8. Роадмап стартапа

Следующие функции

- Уведомления об изменении статуса заявки (email и Telegram-бот): студенту не придётся открывать сайт, чтобы узнать ответ.
- Загрузка PDF-резюме и разбор навыков из него.
- Кабинет вуза: отчёты о востребованности навыков и трудоустройстве студентов (платный продукт).
- Модерация вакансий и интерфейс администратора, подтверждение email, сброс пароля, ограничение числа попыток входа.
- Полнотекстовый поиск по описаниям вакансий (`pg_trgm`).

- Платное продвижение вакансий и подписка для компаний (монетизация).

Как масштабировать

- **События** (самая быстрорастущая коллекция): индекс по времени с TTL или разбиение по месяцам, архив старых данных; запись через очередь, чтобы пик трафика не давил на базу.
- **PostgreSQL:** реплики на чтение для каталога и дашборда, отдельный пул для записи; кэш каталога (список вакансий мало меняется); индексы подбираем по `EXPLAIN ANALYZE` на реальном трафике.
- **Приложение:** несколько экземпляров за балансировщиком (приложение не хранит состояния: сессия в JWT-cookie); статические файлы через CDN.
- Платные тарифы вместо бесплатных (Render не засыпает, Supabase не приостанавливается).

Что анализировать при миллионе пользователей

- **Воронка по когортам:** просмотр → отклик → собеседование → оффер → принятие; где теряются студенты (по вузам, направлениям, городам).
- **Время до оффера** и доля ответов компаний на заявки: качество работодателей, рейтинг по скорости ответа.
- **Тренды спроса на навыки** по месяцам и регионам: рост и падение технологий; сопоставление с обучением в вузах (продукт для университетов).
- **Рекомендации:** учёт не только навыков, но и истории откликов и принятых офферов; проверка, насколько процент совпадения предсказывает оффер.
- **Качество данных:** дубликаты вакансий, «мёртвые» заявки, подозрительная активность.

Итог

Shogird показывает прозрачный путь студента от поиска до принятого оффера, не допускает двойной брони и перепродажи последнего места благодаря транзакциям и ограничениям PostgreSQL, а MongoDB хранит поток событий и поиски, из которых строится аналитика «разрыва навыков». Проект развёрнут в облаке и доступен по публичной ссылке.